AFTERIMAGE: HUMAN-MEMORY DOCUMENT RETRIEVAL SYSTEM

A PROJECT REPORT

Submitted by

SOURYAM BANDYOPADHYAY (25BCS10077)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



Chandigarh University

November 2026



BONAFIDE CERTIFICATE

Certified that this project report **"AFTERIMAGE: HUMAN-MEMORY DOCUMENT RETRIEVAL SYSTEM"** is the bonafide work of **"SOURYAM BANDYOPADHYAY"** who carried out the project work under my supervision.

SIGNATURE **SIGNATURE**

SUPERVISOR

HEAD OF THE DEPARTMENT

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

List of Figures 4
List of Tables 5
List of Standards 6
CHAPTER 1.INTRODUCTION 11
1.1. Identification of Client / Need / Relevant Contemporary issue 11
Identification of Problem 11
1.2. Identification of Tasks 11
Timeline 12
1.3. Organization of the Report 13
CHAPTER 2.LITERATURE REVIEW/BACKGROUND STUDY 14
1.4. 2.1 Timeline of the reported problem 14
Existing solutions 14
1.5. 2.2 Bibliometric analysis 15
Review Summary 15
2.3. Problem Definition 16
Goals/Objectives 16
CHAPTER 3.DESIGN FLOW/PROCESS 17
3.1. 2.4 Evaluation & Selection of Specifications/Features 17
2.5. Design Constraints 18
3.2. Analysis of Features and finalization subject to constraints 18
2.6. Design Flow 19
3.3. Design selection 20
Implementation plan/methodology 31
3.4. **CHAPTER 4.RESULTS ANALYSIS AND VALIDATION 31**
4.1. Implementation of solution 38
3.5. **CHAPTER 5.CONCLUSION AND FUTURE WORK 38**
5.1. Conclusion 38
3.6. Future work 40
5.2. **REFERENCES 41**
APPENDIX 41
1. Plagiarism Report 42
Design Checklist 43
2. **USER MANUAL ..... **

List of Figures

Figure 1.1: Gantt chart of the project plan (weeks) 1
Figure 3.1: Architecture of AFTERIMAGE: web layer, C++ engine (indexing path and search path) and SQLite 3
Figure 3.2: Flow of a search: query understanding, five retrieval routes, ranking, graph and hybrid re-ranking 25
Figure 3.3: Bipartite document–concept graph (example from the sample library) 2
Figure 3.4: Stages of the query parser 6
Figure 4.1: Benchmark: queries returning results and average time per query 2
Figure 4.2: Memory Search showing the interpretation bar, 98% match, page, highlighted passage and “Why selected” explanation 7
Figure 4.3: Typo-tolerant search: the query “schedulng” is matched to “scheduling” 3
Figure 4.4: Developer Console: library statistics (documents, vocabulary, postings, graph concepts and edges) and tools 4
Figure UM.1: Home screen of the web interface with sample recollections to try 5
Figure UM.2: Search result card with highlighted passage and explanation 6
Figure UM.3: Library view listing the indexed documents 4
Figure UM.4: Developer console and SQL workbench 6
4
6
4
7

List of Tables

Table 1.1: Tasks required to identify, build and test the solution 1
Table 2.1: Milestones in the history of the problem 2
Table 2.2: Analysis of existing solutions 1
Table 2.3: Project objectives and how each is validated 4
Table 3.1: Evaluation of candidate features 1
Table 3.2: Design constraints and their effect on the design 5
Table 3.3: Comparison of the alternative designs 1
Table 3.4: The eight engine modules 6
Table 3.5: Extraction method for each input type 1
Table 3.6: Retrieval routes, weights and conditions 7
Table 3.7: Time expressions understood by the parser 1
Table 3.8: Signals used in hybrid re-ranking 8
Table 3.9: Endpoints of the local web adapter 1
Table 3.10: Time complexity of the main operations 9
Table 4.1: Automated test groups and what each proves 2
Table 4.2: Score breakdown for the query “raft leader heartbeat” 0
Table 4.3: Benchmark results 2
Table 4.4: Objectives against evidence 2
Table UM.1: Common problems and fixes 2
4
2
8
2
8
2
9
3
0
3
1
3
3
3
4
3
6
4
7

List of Standards

Standard	Publishing Agency	About the standard	Page no. where used
ISO/IEC 14882:2020	ISO/IEC	Programming languages: C++. The C++20 language and library (std::filesystem, string_view, chrono) in which the search engine is written.	21, 23
IEEE Std 1003.1 (POSIX)	IEEE / The Open Group	Portable operating system interface. posix_spawn, pipe, fcntl, kill and waitpid are used to run Poppler and Tesseract without a shell.	22
ISO/IEC 9075	ISO/IEC	Database language SQL. SQLite implements the SQL used for the documents table, upserts and transactions.	22
ISO 32000-2	ISO	Portable Document Format (PDF). The format whose text layer and page images are extracted through Poppler.	22
RFC 3629 (UTF-8)	IETF	UTF-8 encoding. The tokeniser treats bytes of value 128 or more as part of a word so UTF-8 words are not split.	23
RFC 8259 (JSON)	IETF	JSON data interchange format. Used between the C++ engine (search --json) and the Python web adapter.	28
RFC 9110 (HTTP Semantics)	IETF	HTTP methods and status codes used by the local REST-style API of the web interface.	28
ISO/IEC 25010:2011	ISO/IEC	Systems and software quality models. Functional suitability, performance efficiency, security and maintainability are the quality attributes used to evaluate the project.	36
ISO/IEC/IEEE 29119	ISO/IEC/IEEE	Software testing. Unit, integration and command-line system tests were organised in line with its test levels.	31

ABSTRACT

People rarely remember the exact name or exact words of a file they saved months ago. They remember a topic, a few words, a rough time, or the look of a page. Ordinary file-name search and plain keyword search both fail in that situation, and cloud-based assistants solve it only by uploading private documents to a remote service.

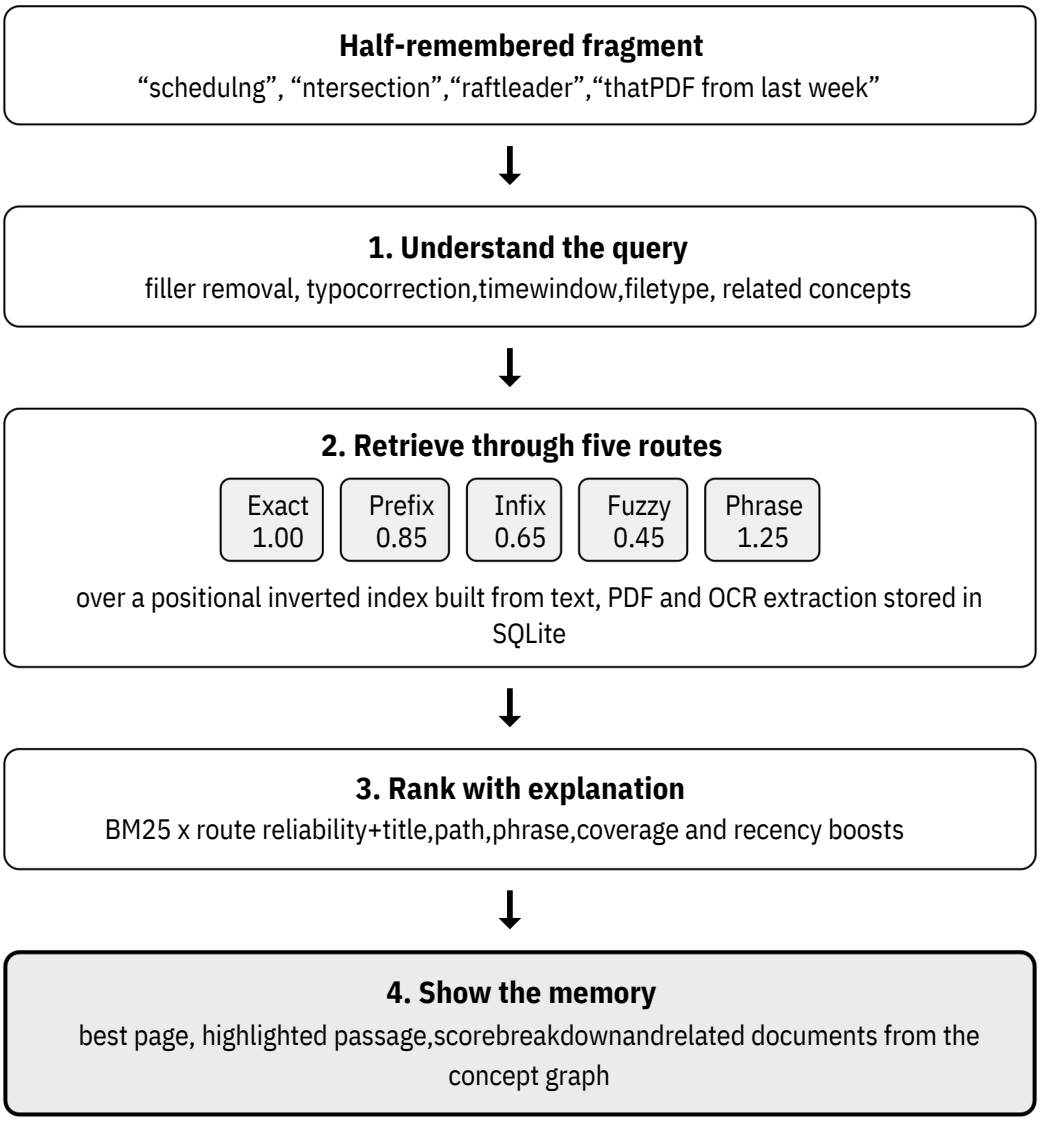
This project presents AFTERIMAGE, a local-first document retrieval system that finds a stored document from an imperfect fragment of memory. The core engine is written in C++20 and is built as eight cooperating modules: document model, folder ingestion, content extraction (plain text, Markdown, PDF text, scanned PDF and image OCR through Poppler and Tesseract), SQLite persistence, a positional inverted index, a fragment retrieval engine, a ranking engine and a document-concept memory graph. Retrieval combines five routes: exact, prefix (ordered-map range search), infix (middle-of-word), typo-tolerant (bounded Levenshtein distance) and phrase (position-based). Candidates are ranked with the BM25 model multiplied by a per-route reliability factor, plus explicit title, path, phrase, coverage and recency boosts, so every result carries a score breakdown. A breadth-first search over the concept graph then suggests related documents.

A Python layer adds natural-language query understanding (filler removal, typo correction, time expressions such as “last week”, file-type words and concept expansion), hybrid re-ranking, page-level highlighted passages, optional local semantic search with Ollama embeddings, and a browser interface with a read-only developer console.

The system was verified with 12 automated test groups (all passing, about 290 assertions) and a 50-query recollection benchmark. On the bundled corpus the engine answers in about 4 ms per query, and on a library of 2,185 documents in about 60 ms per query. All processing stays on the user’s computer.

Keywords: information retrieval, inverted index, BM25, edit distance, OCR, SQLite, memory graph, local-first software.

GRAPHICAL ABSTRACT



Everything runs on the user’s own computer: C++ engine, SQLite database, local web interface.

ABBREVIATIONS

Abbreviation	Full form
BFS	Breadth-First Search
BM25	Best Matching 25 (Okapi) ranking function
CLI	Command-Line Interface
CMake	Cross-platform build system generator
CRUD	Create, Read, Update, Delete
DP	Dynamic Programming
HTTP	HyperText Transfer Protocol
IDF	Inverse Document Frequency
JSON	JavaScript Object Notation
OCR	Optical Character Recognition
PDF	Portable Document Format
POSIX	Portable Operating System Interface
RAII	Resource Acquisition Is Initialization
SQL	Structured Query Language
STL TF	Standard Template Library
UI	Term Frequency
UTF-8	User Interface
	8-bit Unicode Transformation Format

SYMBOLS

Symbol	Meaning
N	Number of documents in the library
n_t	Number of documents containing term t (document frequency)
$f_{t,d}$	Count of term t in document d (term frequency)
$\|d\|$	Length of document d in tokens
avgdl	Average document length in the library
k_1	BM25 term-frequency saturation constant (1.2)
b	BM25 length-normalisation constant (0.75)
ρ_t	Route reliability of a match: exact 1.0, prefix 0.88, infix 0.72, fuzzy 0.58
H_{title}, H_{path}	Number of matched terms found in the title, or only in the path
C	Query coverage: matched query words divided by total query words
D(i, j)	Levenshtein distance between the first i characters of one word and the first j characters of another
V	Vocabulary size (number of unique terms)
L	Typical word length in characters
P	Number of positions of a term in a document
df(c)	Number of documents that use concept c
hops(c)	Number of graph edges between a seed document and a result through concept c
q, **d**	Embedding vectors of the query and of a document

CHAPTER 1.
INTRODUCTION

1.1. Identification of Client / Need / Relevant Contemporary issue

The client for this project is an individual student or professional who keeps a large personal library of study notes, source files, PDFs, scanned pages and screenshots on a local computer. Digital storage is cheap, so such libraries keep growing, but the time to find one specific file does not shrink. The contemporary issue is that most people can no longer find their own information by its name or exact keyword, and the usual cure, a cloud AI assistant, requires uploading private documents to a remote service.

The need was justified in three ways. No external statistical survey was conducted, so no survey figures are claimed.

- **Real library.** The author's own working library, indexed during development, contained 2,185 documents (about 12.8 million characters of extracted text) made up of Markdown, plain text, PDF and image files. Finding anything in a collection of this size by remembering a file name is not realistic.
- **Realistic recollection patterns.** A benchmark of 50 human-style queries was written (vague recollections, different wording, messy conversational sentences, PDF and scan queries, and relationship queries) to represent how people actually describe a half-remembered file (Chapter 4).
- **Privacy.** The documents include coursework, a résumé and scanned certificates. Searching them must not depend on a third-party server.

1.2. Identification of Problem

A person wants to open a document they saved earlier but can recall it only as a fragment: a few letters of a word, a spelling that is slightly wrong, a phrase from inside the text, a topic, a rough time, or the type of file. Searching by file name fails because the name is forgotten. Searching by exact keyword fails because the typed word is wrong, partial or different from the word in the file. Text that exists only inside scanned pages and images cannot be searched at all. Finally, a flat list of matches does not tell the person why a file matched, where inside it to look, or which other files are related.

1.3. Identification of Tasks

The work was divided into the tasks below, which also define the framework of this report.

Table 1.1: Tasks required to identify, build and test the solution

No.	Task	Output	Report chapter
1	Study the problem and existing search techniques	Literature review and problem definition	Chapter 2
2	Choose features, constraints and the best design among alternatives	Feature list, constraint analysis, design comparison	Chapter 3
3	Build the document model, folder ingestion and storage	Core, ingestion and SQLite modules	Chapter 3
4	Extract text from TXT, Markdown, PDF and images (OCR)	Extraction module	Chapter 3
5	Build the positional inverted index and the five retrieval routes	Indexing and retrieval modules	Chapter 3
6	Rank results and explain scores; build the concept graph	Ranking and memory-graph modules	Chapter 3
7	Add natural-language understanding and a browser interface	Python adapter and web UI	Chapter 3
8	Test every module, run the benchmark and analyse results	Test suite, benchmark, analysis	Chapter 4
9	Conclude, list limitations and plan future work	Conclusion and future work	Chapter 5

1.4. Timeline

Figure 1.1 shows the planned project timeline as a Gantt chart. Weeks are counted from the start of the project; the bars overlap where one activity began before the previous one finished.

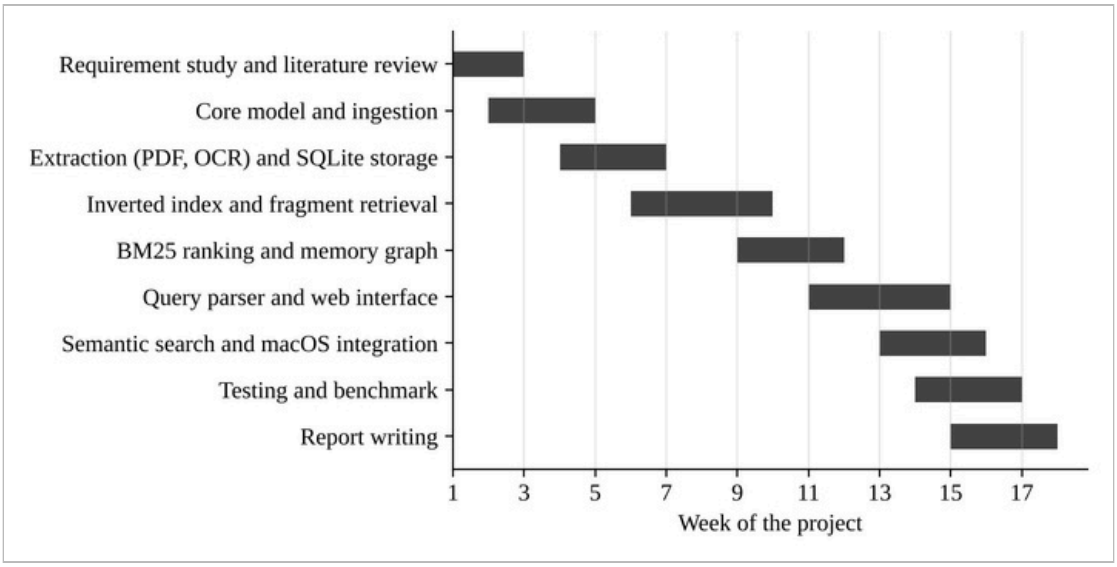


Figure 1.1: Gantt chart of the project plan (weeks)

1.5. Organization of the Report

- **Chapter 1 (Introduction)** identifies the need, defines the problem, lists the tasks and gives the timeline.
- **Chapter 2 (Literature Review / Background Study)** traces the history of the problem, reviews existing solutions, compares them and states the problem definition and objectives.
- **Chapter 3 (Design Flow / Process)** evaluates features, lists constraints, compares three alternative designs, selects one, and describes the implementation methodology, data structures and algorithms in detail.
- **Chapter 4 (Results Analysis and Validation)** presents the tools used, the automated tests, sample outputs, the 50-query benchmark and an analysis of results.
- **Chapter 5 (Conclusion and Future Work)** summarises outcomes, deviations from expectations and the way ahead.
- The report ends with the References, the Appendix (plagiarism report and design checklist) and the User Manual.

CHAPTER 2.
LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the reported problem

Finding stored information from an imperfect description is one of the oldest problems in information retrieval. Table 2.1 lists the main milestones that this project builds on. Years are those commonly cited in the literature; where a year is approximate it is marked with "c.".

Table 2.1: Milestones in the history of the problem

Period	Milestone	Relevance to this project
1966	Levenshtein defines the edit distance between two strings [3]	Basis of typo-tolerant matching
1960s–70s	Salton's SMART system and the vector-space model; Spärck Jones introduces inverse document frequency in 1972 [4]	Idea that rare words are more informative
c. 1994	Okapi BM25 ranking is developed and evaluated at TREC-3 [7]	Ranking model used in this project
2004–2005	Desktop search tools appear for personal computers (for example Apple Spotlight in 2005)	File and keyword search on the local machine
2005	The Tesseract OCR engine is released as open source [6]	Makes scanned text searchable locally
2009	Robertson and Zaragoza publish a survey of BM25 and its variants [1]	Source of the BM25 formulation and parameters
2013 onward	Neural word and sentence embeddings become practical for semantic search	Optional semantic layer in this project
2023 onward	Local model runners such as Ollama allow embeddings without a cloud service [11]	Private, offline semantic search

2.2. Existing solutions

Five families of existing solutions are relevant. Each is described briefly below.

- **Operating-system search (file name and indexed content).** Tools such as Apple Spotlight index file names, metadata and the text of common formats and respond instantly.
- **Command-line text search (grep and similar tools).** These scan files for an exact string or regular expression.

- **Search libraries and servers (for example Apache Lucene and Elasticsearch).** These provide inverted indexes, BM25 ranking, fuzzy queries and highlighting, but are designed as general search infrastructure.
- **Embedded full-text search (SQLite FTS5).** SQLite offers tokenised full-text tables with prefix and phrase queries and a bm25() function inside the database.
- **Cloud AI and vector search.** Hosted assistants and vector databases match by meaning using embeddings.

2.3. Bibliometric analysis

Table 2.2 analyses the solutions on key features, effectiveness and drawbacks, from the point of view of the problem in Section 1.2. The assessment is the author's own, based on the documented behaviour of each family of tools.

Table 2.2: Analysis of existing solutions

Solution	Key features	Effectiveness for the problem	Drawbacks
OS search	Instant, integrated, file name and content	Good when the word is exact	Little tolerance for typos or fragments; limited explanation of why a file matched
grep-style tools	Exact or regex scan, no setup	Good for exact code or text	Re-reads every file for each query; no ranking, no typo tolerance, no OCR
Lucene / Elasticsearch	Inverted index, BM25, fuzzy queries, highlights	Very effective and scalable	Heavy to install and operate for one person; no built-in personal-memory features such as time phrases or related-document graph
SQLite FTS5	Embedded, prefix and phrase queries, bm25()	Effective for tokenised keyword search	No built-in edit-distance matching; ranking cannot be split into project-specific boosts
Cloud AI / vector search	Meaning-based matching, natural questions	Effective for paraphrase	Private files leave the computer; scores are hard to explain; needs a model call for every document and query

2.4. Review Summary

The review shows that every technique needed to solve the problem already exists, but no single everyday tool combines them for a private, personal library: exact and fragment matching, typo tolerance, search inside scans, an explanation of each result, awareness of time words such as "last week", and a way to discover related files. The project therefore does not invent new retrieval theory. It selects proven techniques (inverted index, ordered- vocabulary range search, edit distance, BM25, breadth-first search) and integrates them into one local system whose behaviour can be inspected and tested.

2.5. Problem Definition

What is to be done. Design and build a local software system that, given an imperfect fragment of memory, returns the intended document from a personal library of text, PDF and image files, ranks it high, explains why it matched and suggests related documents. **How it is to be done.** By scanning folders, extracting text (using OCR where needed), storing it in a database, building an in-memory positional inverted index, retrieving candidates through exact, prefix, infix, typo-tolerant and phrase routes, ranking them with BM25 and explicit boosts, and linking documents through a concept graph. A web interface and a command-line interface expose the engine.

What is not to be done. The system will not upload documents to any server, will not modify or delete the user's files, will not attempt handwriting recognition or image understanding beyond OCR text, and will not be a multi-user or network search service.

2.6. Goals/Objectives

The goals below are narrow, specific, tangible and can be validated or measured.

Table 2.3: Project objectives and how each is validated

No.	Objective	Validation
O1	Store documents persistently with stable IDs	Storage and document-manager unit tests; reload test
O2	Extract text from TXT, Markdown, PDF text layers, scanned PDFs and images	Extraction tests with PDF and PNG fixtures
O3	Implement a positional inverted index with prefix, infix, fuzzy and phrase lookup	Indexing and retrieval tests
O4	Return a document from a one-letter typo, a stem or a middle-of-word fragment	Sample queries "schedulng" and "ntersection"
O5	Rank with BM25 and show a score breakdown for each result	Ranking tests; breakdown printed by CLI and UI
O6	Suggest related documents from shared rare concepts	Graph tests; related list in sample outputs
O7	Understand natural queries and time expressions	50-query benchmark; interpretation bar in UI
O8	Keep all data local and respond in well under one second	Design review; measured 4 ms and 60 ms per query

CHAPTER 3.
DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

The features found in the literature were evaluated against the problem definition. Each feature was marked as required, desirable or rejected.

Table 3.1: Evaluation of candidate features

Feature (source)	Decision	Reason
Inverted index with positions (Lucene, FTS5)	Required	Fast lookup; positions give exact phrase search
Prefix search on an ordered vocabulary	Required	Handles stems and partial words cheaply
Infix (middle-of-word) search	Required	Handles fragments such as “ntersection”
Edit-distance fuzzy search (Lucene)	Required	Handles misspellings
BM25 ranking (Okapi, Lucene)	Required	Standard, well-understood relevance model
Explainable score breakdown	Required	The person must see why a file matched
OCR for scans and images (Tesseract)	Required	Makes locked text searchable
Concept graph of related documents	Required	Discovers files the user did not ask for
Natural-language time and type words	Desirable	Matches how people describe memories
Local semantic embeddings	Desirable (optional)	Matches meaning; must not be mandatory
Distributed or multi-user search	Rejected	Outside the scope of a personal tool
Cloud APIs	Rejected	Conflicts with the privacy constraint

3.2. Design Constraints

3.2.1 Standards:

Regulations, economic, environmental, health, manufacturability, safety, professional, ethical, social and political issues and cost were considered in the design. The standards relied on are listed in the List of Standards. Table 3.2 states how each constraint shaped the design.

Table 3.2: Design constraints and their effect on the design

Constraint	Consideration	Effect on the design
Regulations and privacy	Personal documents contain private information	All processing is local; the web server binds to 127.0.0.1 only; no telemetry
Economic and cost	A student project must be free to build and run	Only open-source components: C++, SQLite, Poppler, Tesseract, Python standard library
Environmental	Avoid wasteful computing	No GPU; incremental indexing skips unchanged files; semantic layer is optional
Health and safety	Running external programs on untrusted files	No shell, time limit, output cap and size limits for external tools; symbolic links not followed
Manufacturability / maintainability	The code must be modifiable and testable	Eight modules with headers, one test file per module, CMake and CTest
Professional and ethical	Respect the user's files	Read-only access to source files; developer console allows only SELECT, PRAGMA and EXPLAIN
Social and political	Users with different languages	UTF-8 aware tokeniser; selectable Tesseract language (for example hin+eng)

3.3. Analysis of Features and finalization subject to constraints

- **Modified:** semantic search was kept but made optional and local-only (Ollama), because the privacy and cost constraints rule out cloud embeddings.
- **Modified:** fuzzy matching was restricted to words of four or more letters and made a fallback, to avoid noisy results and wasted work.
- **Added:** extraction limits (50 MiB input, 12 MiB text, 20 OCR pages, 120 s per tool) and a no-shell process runner, because of the safety constraint.
- **Added:** a dependency-noise penalty in the web layer, because real libraries contain many documentation files from software dependencies.
- **Removed:** any feature that needs a network connection or a server process other than the local web adapter.

3.4. Design Flow

Three alternative designs were considered for the retrieval core.

Design A: database-only search. Store text in SQLite and search with SQL LIKE or an FTS5 virtual table. The database performs tokenising, matching and bm25() ranking. The application only formats results.

Design B: custom positional index over a database store (hybrid). SQLite stores the extracted text as the source of truth. A C++ engine rebuilds a positional inverted index (ordered map) and a concept graph in memory, retrieves through five routes, ranks with

BM25 and explicit boosts, and a Python layer adds query understanding, hybrid re-ranking and the user interface.

Design C: embedding-only search. Convert every document and every query into an embedding vector with a language model and return the nearest vectors by cosine similarity.

3.5. Design selection

The designs were compared on the criteria of Table 3.3. Ratings from 1 (poor) to 5 (excellent) were assigned by the author from the documented behaviour of each approach; the weights reflect the objectives in Section 2.6.

Table 3.3: Comparison of the alternative designs

Criterion (weight)	Design A: SQL / FTS5	Design B: custom index + SQLite	Design C: embeddings only
Typo and fragment tolerance (5)	2	5	3
Explainable ranking (5)	2	5	1
Works offline with no model (4)	5	5	1
Phrase and position control (3)	4	5	1
Meaning-based matching (2)	1	3 (optional layer)	5
Simplicity of build (3)	5	3	3
Educational value: own data structures (3)	1	5	2
Weighted total (maximum 125)	**76**	**112**	**59**

Selection and reason. Design B scores highest and was selected. It is the only design that meets the typo-tolerance, explainability and offline objectives together, it keeps SQLite as a reliable source of truth, and it lets the optional semantic layer from Design C be added without making a language model mandatory. Its disadvantage, building the index in memory for each search session, was accepted because the measured cost is small (Section 4.1.4).

3.6. Implementation plan/methodology

3.6.1 Development approach

The system was built module by module, bottom-up: document model, ingestion, extraction, storage, index, retrieval, ranking, graph, command-line interface, and finally the Python adapter and browser interface. Each module has a public header under include/afterimage/<module>/ and a unit-test file, so it could be tested before the next module depended on it. Derived data (the index and graph) is never stored; it is rebuilt from SQLite, so it cannot disagree with the stored text.

3.6.2 System architecture

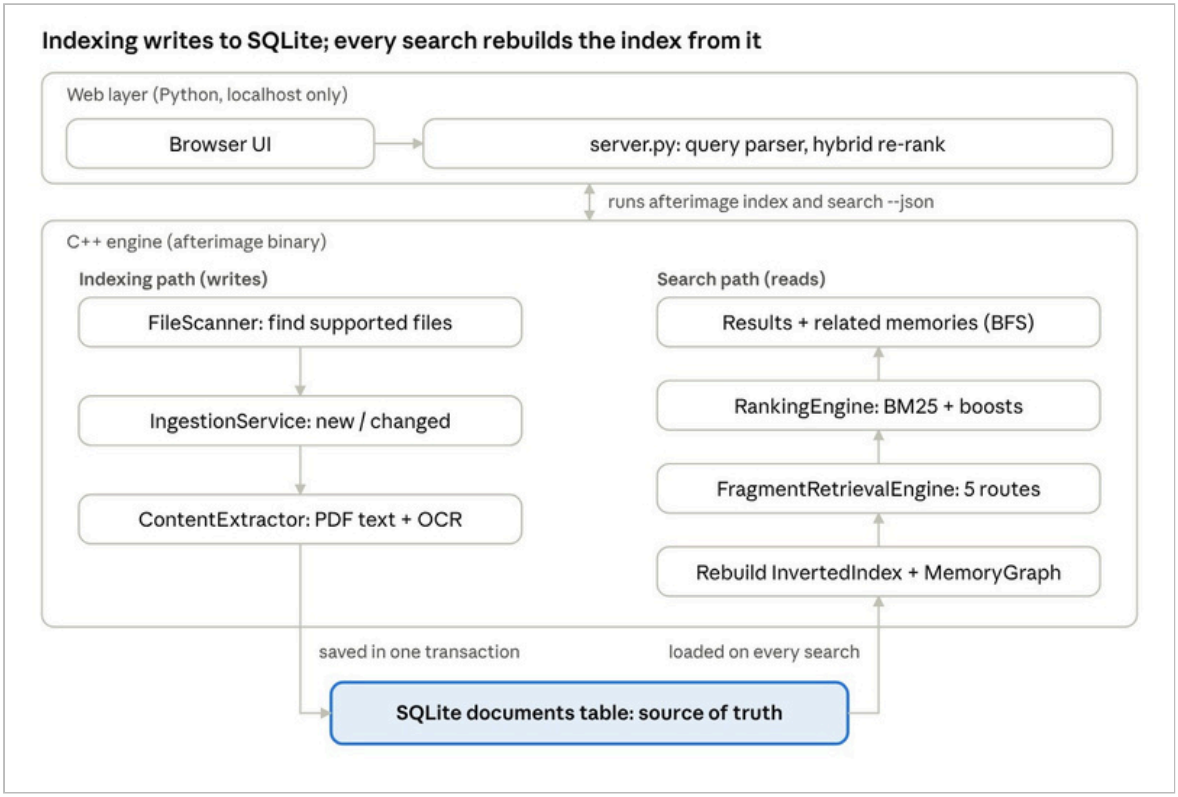


Figure 3.1: Architecture of AFTERIMAGE: web layer, C++ engine (indexing path and search path) and SQLite

The C++ engine is usable alone from the terminal. The Python adapter calls the engine as a subprocess (afterimage index and afterimage search --json), adds query understanding and hybrid ranking, and serves the browser interface. SQLite is the only place where the indexing path and the search path meet.

Table 3.4: The eight engine modules

No.	Module	Source files	Responsibility
1	Core document model	core/DocumentManager.cpp	Document record, stable IDs, in-memory library

2	Ingestion	ingestion/FileScanner.cpp, IngestionService.cpp	Recursive scan, filtering, duplicate and change detection
3	Extraction	extraction/ContentExtractor.cpp, ExtractionService.cpp, TextNormaliser.cpp	Text, PDF and OCR extraction with safety limits
4	Persistent storage	storage/SQLiteDocumentStore.cpp	SQLite save and load with transactions
5	Text indexing	indexing/Tokeniser.cpp, InvertedIndex.cpp	Tokens, postings with positions, prefix, infix, fuzzy, phrase lookup
6	Fragment retrieval	retrieval/FragmentRetrievalEngine.cpp	Combines the five routes into weighted candidates with evidence
7	Ranking	ranking/RankingEngine.cpp	BM25 plus title, path, phrase, coverage and recency boosts
8	Memory graph	graph/MemoryGraph.cpp	Document-concept graph and related-document search

The project contains about 2,900 lines of C++ source, 740 lines of C++ headers, 1,175 lines of C++ tests, 1,530 lines of Python and about 2,500 lines of web interface code.

```
AFTERIMAGE/
  CMakeLists.txt          build and 12 registered tests
  include/afterimage/     public headers, one folder per module
  src/                    core, ingestion, extraction, storage, indexing,
                          retrieval, ranking, graph, main.cpp (CLI)
  web/                    server.py (HTTP + query parser), app.js, index.html,
style.css
  macos                   AfterimageLauncher.swift, AfterimageSpotlight.swift
  / tests/                unit/, integration/, fixtures/ (PDF, PNG),
test_50_queries.py
  data/                   sample_documents/, advanced_test_corpus/, test_corpus_50/
```

3.6.3 Core document model and ingestion

A Document holds id, title, file path, file type, extracted text, size and modification time. DocumentManager stores documents in a vector and keeps an unordered_map from ID to position for O(1) lookup. IDs come from a counter; when the library is reloaded from SQLite, restoreDocument keeps the saved ID and moves the counter past it, so IDs are stable across restarts and are never reused. Insertion is exception-safe: the lookup entry is rolled back if the vector insertion throws.

The FileScanner uses std::filesystem::recursive_directory_iterator with skip_permission_denied. Symbolic links are counted and skipped, hidden entries (names starting with a dot) are skipped and hidden folders are not entered. Only the extensions txt, md, markdown, pdf, png, jpg, jpeg, tif, tiff, bmp, webp and gif are kept. Every problem becomes a ScanIssue in the report instead of stopping the scan. The IngestionService normalises each path with weakly_canonical so one file reached by two routes is one document, and classifies every file as new, already known, changed (size or modification

time differs) or unsupported. Only new and changed files are sent for extraction, so slow OCR is never repeated needlessly; the flag --force re-extracts everything.

3.6.4 Content extraction

Table 3.5: Extraction method for each input type

Input	Method	Fallback or limit
TXT, MD	Read bytes, strip UTF-8 byte-order mark, convert line endings	50 MiB input limit, 12 MiB text limit (status Partial)
PDF with text layer	pdftotext with UTF-8 and layout mode	120 s timeout
Scanned PDF	If no text is returned: pdftoppm renders pages to PNG (max 1800 px), Tesseract reads each page	First 20 pages; status Partial if more exist
Images	Tesseract with selectable language, page-segmentation mode 3	120 s timeout

External tools run through a small runProcess function built on posix_spawn with a pipe for output. It never builds a shell command, so a file name containing spaces or shell characters cannot inject commands. It reads the pipe without blocking, caps the output size and kills the child with SIGKILL on timeout. Every outcome is an ExtractionStatus (Success, Partial, Empty, Unsupported, DependencyMissing, Failed or TooLarge). ExtractionService wraps each file in try/catch so one bad file never aborts a run. TextNormaliser removes the byte-order mark, unifies line endings and trims trailing blanks.

3.6.5 Persistent storage

```
CREATE TABLE documents (
    id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    file_path TEXT NOT NULL UNIQUE,
    file_type TEXT NOT NULL,
    extracted_text TEXT NOT NULL DEFAULT '',
    size_bytes INTEGER,
    modified_at_epoch_seconds INTEGER
);
CREATE INDEX idx_documents_file_type ON documents(file_type);
```

The web layer adds the columns tags, user_notes, extraction_method, page_count and indexed_at_epoch_seconds, and a second table semantic_vectors (document_id, content_hash, dimensions, vector BLOB, indexed_at). Implementation points: a RAII class Statement prepares a statement in its constructor and finalises it in its destructor, so a thrown error cannot leak a prepared statement; all values are passed through bound parameters (sqlite3_bind_*) and never pasted into SQL; documents are saved with INSERT … ON CONFLICT(id) DO UPDATE; saveAll runs BEGIN IMMEDIATE, saves every document and COMMITs, and ROLLBACKs on any exception; a busy timeout of 5

seconds and the full-mutex flag let the command line and the web server share the file; missing size and time values are stored as NULL, not as fake zeros.

3.6.6 Tokenisation and the positional inverted index

Tokenisation (Tokeniser.cpp). The text is scanned one byte at a time. A byte belongs to a word if it is an ASCII letter or digit, or has a value of 128 or more (so UTF-8 words are not cut apart). ASCII letters are lowercased and every other byte ends the current token. Each token is stored with its position counted in words. Example: “Raft elects a Leader.” gives raft(0), elects(1), a(2), leader(3). Cost: O(n) for n bytes.

Inverted index (InvertedIndex.cpp). Instead of reading every document for every query, the index stores a dictionary from each word to the documents that contain it and the positions where it occurs.

```
postingsByTerm_    : std::map<string, vector<Posting>>      (sorted by term)
   Posting         : { documentId, vector<position>}
termsByDocument_ : unordered_map<id, vector<term>>         (for fast removal)
lengthByDocument_ : unordered_map<id, tokenCount>          (needed by BM25)
```

To add a document the index (1) removes any old postings for the same ID so re-indexing never duplicates, (2) tokenises, (3) groups positions by term, (4) appends one Posting per term, and (5) records the document length and updates the running total used for the average length. Adding costs O(n + u log V) for n tokens, u unique terms and vocabulary size V. A std::map is used instead of an unordered_map because sorted terms make prefix search possible without scanning; looking up one term costs O(log V).

3.6.7 Prefix, infix and fuzzy expansion

Prefix search. A query word such as “travers” should find “traversal” and “traversing”. In the sorted vocabulary all words starting with the prefix sit next to each other. The engine calls lower_bound(prefix) to jump to the first candidate and walks forward while the term still starts with the prefix, stopping at 20 expansions. Cost: O(log V + k) for k matches. **Infix search.** For fragments from the middle of a word (“ntersection” finds “intersections”) a sorted map cannot help, so the engine scans the vocabulary with string find. Matches where the fragment appears earlier in the word rank first. It only runs for fragments of 3 or more characters and returns at most 8 terms. Cost: O(V × L).

Fuzzy search. Two words are close if few single-character edits (insert, delete, substitute) turn one into the other. This is the Levenshtein distance, computed by dynamic programming:

$$D(i,j) = \min\{\, D(i-1,j)+1,\ D(i,j-1)+1,\ D(i-1,j-1)+[a_i \neq b_j] \,\} \quad D(i,0)=i,\ D(0,j)=j$$

The implementation (boundedEditDistance) uses three optimisations. A *length filter* returns at once if the word lengths differ by more than the limit (1). It keeps *two rows only*, so memory is O(L) instead of O(L^2). An *early exit* rejects a word pair as soon as the smallest value in a row exceeds the limit. Example: “schedulng” to “scheduling” is one insertion, distance 1. Fuzzy search only runs for words of 4 or more letters.

3.6.8 Phrase search with positions

A phrase such as “raft leader” must appear as consecutive words. The engine takes the posting list of the first word; for each document it fetches the position lists of all phrase words (a missing word means failure); for every start position p of the first word it checks, by binary search, that word k occurs at position p + k. The first start position where all offsets succeed proves the phrase. Cost: O(P_1 × k × log P). Storing positions in the index is what makes this possible.

3.6.9 Fragment retrieval engine

This module does not rank; it gathers candidates and records why each one was found. For every distinct query word it tries several routes, each with a reliability weight, and sums the evidence per document (Table 3.6).

Table 3.6: Retrieval routes, weights and conditions

Route	What it matches	Weight	Condition
Exact	the word itself	1.0	always
Prefix	words starting with it	0	up to 20 expansions
Infix	words containing it	0.8 5	fragment of 3+ characters, up to 8 expansions
Fuzzy	words one edit away	0.45 0.6 5	only if no exact, prefix or infix match was found; 4+ characters
Phrase	all words consecutive	1.25	query has 2+ words

Two design decisions matter. First, fuzzy search is a *fallback*, used only when stronger routes found nothing, so it cannot add noise to a good query. Second, each (route, query word, matched term) triple is stored once, so the same evidence is never counted twice. The result keeps a list of RetrievalMatch records, which becomes the “Matched by” line shown to the user. The index’s own search function scores with a simple weighted tf-idf only to decide candidates; the final order comes from the separate ranking stage, so the formula can change without touching the index.

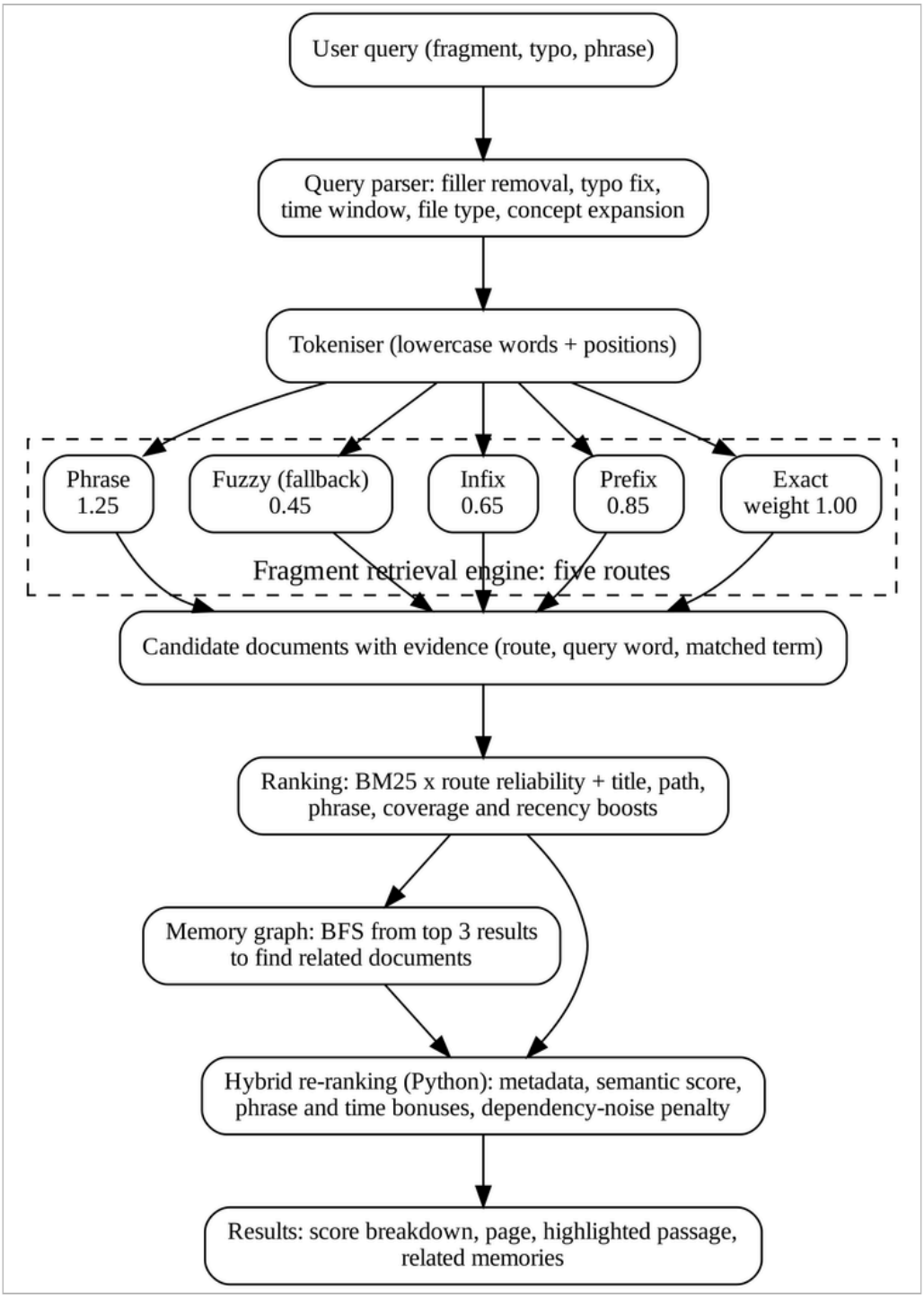


Figure 3.2: Flow of a search: query understanding, five retrieval routes, ranking, graph and hybrid re-ranking

3.6.10Ranking with BM25 and explainable boosts

BM25 is the standard relevance formula in search engines. It rewards words that are frequent in a document but rare in the library, and it stops long documents from winning just because they are long.

$$\mathrm{BM25}(d,q) = \sum_{t \in q} \rho_t \ln\left(1 + \frac{N - n_t + 0.5}{n_t + 0.5}\right) \cdot \frac{f_{t,d}\,(k_1 + 1)}{f_{t,d} + k_1\left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}$$

Here N is the number of documents, n_t the number containing term t, $f_{t,d}$ the term's count in document d, |d| the document length, avgdl the average length, k_1 = 1.2 and b = 0.75. The factor ρ_t is the *route reliability* of the match: 1.0 for exact, 0.88 for prefix, 0.72 for infix and 0.58 for fuzzy, so a typo match contributes less than an exact one. For each matched vocabulary term only the strongest route is counted. The final score adds small named boosts:

$$\mathrm{score} = \mathrm{BM25} + 1.25\,H_{title} + 0.25\,H_{path} + 0.75\,[\mathrm{phrase}] + 0.5\,C + 0.2 \cdot 2^{-\mathrm{age}/90}$$

H_{title} is the number of matched terms found in the title (a term found in the title is not counted again as a path hit), H_{path} the number found only in the file path, [phrase] is 1 if the whole query occurs as a consecutive phrase, C is query coverage (matched query words divided by total query words), and the last term is recency, which halves every 90 days since the file was modified and can be switched off with --no-recency. Each component is stored in a RankingBreakdown, so the command line and the web interface can print the explanation for any result.

3.6.11Memory relationship graph

Two documents are probably related if they share rare, informative words. The graph is bipartite: document nodes on one side, concept nodes on the other, with an edge when a document uses a concept (Figure 3.3).

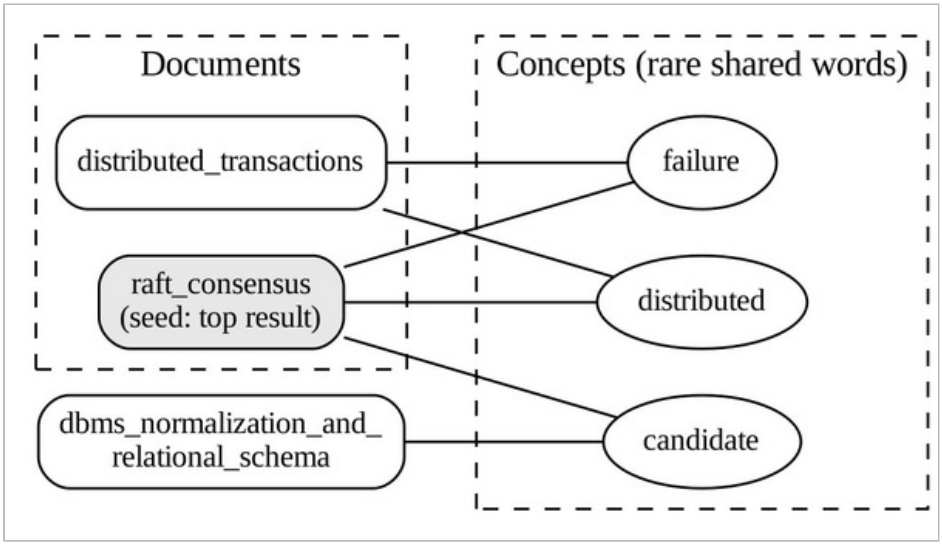


Figure 3.3: Bipartite document–concept graph (example from the sample library)

Building. (1) For each document collect candidate concepts from the title and body; words shorter than 4 letters and about 200 stop-words (for example "system", "data", "process")

are dropped. (2) Keep a body word only if it appears at least twice or in the title. (3) Weight = frequency + 2 × title frequency; keep the top 120 concepts per document. (4) Remove concepts used by more than 25 percent of the library (when there are at least 2 documents), because such hub words connect everything and mean nothing.

Searching. The top 3 search results are the seeds. A breadth-first search with a queue walks seed, concept, neighbouring document; each such step counts as 2 edges, and with the default limit of 2 hops the results are documents sharing at least one concept with a seed. Rare shared concepts count more, and longer paths count less:

$$\mathrm{idf}(c) = \ln\left(1 + \frac{N}{df(c)}\right) \qquad \mathrm{score}(d) = \sum_{c \in \mathrm{shared}(d)} \frac{\mathrm{idf}(c)}{\mathrm{hops}(c)/2}$$

The five strongest connecting concepts are shown as the explanation (for example "Connected through: distributed, failure"). The graph is a lexical heuristic, not a knowledge graph: a shared word is evidence of a relationship, not proof. Cost: building O(N × T) with T at most 120 concepts per document; searching is bounded by the 2-hop limit.

3.6.12 Natural-language query understanding and hybrid ranking

The Python class MemoryQueryParser turns a human sentence into a structured query (Figure 3.4). For "A java assignmnet that i opened last week" it produces the core terms [java, assignment], the corrected typo and a time window.

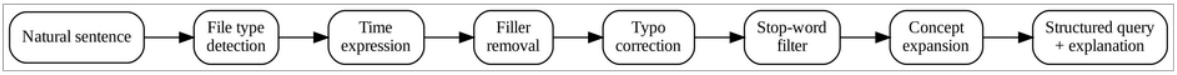


Figure 3.4: Stages of the query parser

1. **File-type detection.** A regular expression finds extensions or media words (pdf, png, jpg, md, txt, images, scan) and sets a file-type constraint.
2. **Time detection.** Expressions are mapped to epoch-second windows (Table 3.7) and removed from the text.
3. **Filler removal.** About 20 patterns strip conversational words ("I remember seeing", "find that", "hey, can you").
4. **Typo correction.** A dictionary of common misspellings maps "schedulng" to "scheduling"; the C++ fuzzy route handles other one-letter typos.
5. **Stop-word filter** leaves the core terms.
6. **Concept expansion.** Synonyms (resume to cv), a small concept taxonomy ("authentication" to identity, password, mfa) and rule-based phrases add candidate words; they never replace the user's own words.
7. **Explanation.** The parser builds the readable sentence shown above the results.

Table 3.7: Time expressions understood by the parser

Expression	Window used	Time weight
2026-03-14 (exact date)	that calendar day	4.0
yesterday, today	last 2 days / last 1 day	4.0
last week	previous Monday to Sunday	3.5
last month	previous calendar month	3.0
recently, a few days ago	last 5 days	3.0
in 2025	that calendar year	2.5
past week, past month	rolling 7 / 30 days	3.5 / 3.0

Hybrid re-ranking. The web layer runs the C++ engine with --json, then merges three candidate sources: the engine's lexical results, a direct SQLite check of title, path, tags and notes (title 4.5, tags 4.0, notes 3.5, path 3.0 points) and local-AI semantic matches when an embedding index exists. If the literal words find nothing, the engine is called again with up to 6 expansion words. Each candidate score is then adjusted as in Table 3.8. The best result is displayed as about 98 percent and others are scaled against it, clamped between 45 and 99 percent; this is a relative display value, not a probability.

Table 3.8: Signals used in hybrid re-ranking

Signal	Effect on score
Engine score (BM25 and boosts)	base value
Semantic cosine similarity	plus 28 × similarity
Fraction of core terms present as whole words	plus 35 × fraction
All terms present (2+ terms)	plus 25
Exact multi-word phrase in text	plus 40
Date inside requested window	plus 18 × window weight; files outside the window are dropped
File type does not match request	file is dropped
Related-concept hits in text	plus 2.2 per hit, capped at 12
Path inside node_modules, _deps, .git or cmake-build	multiplied by 0.12 (dependency noise)

3.6.13Passages, semantic search, web adapter and interface

Page and passage extraction. To show where to look, each candidate's text is split into pages (on form-feed characters for PDF text, otherwise virtual pages of 2,500 characters). A page scores 3 points per query-term occurrence plus 1 per related concept; the best page wins, and a 280-character window starting 80 characters before the first hit is cut, HTML-escaped and highlighted.

Semantic search. For each document the text “Title … Content …” (first 12,000 characters) is hashed with SHA-256; if the hash differs from the stored one, the local Ollama model nomic-embed-text returns a vector (batches of 8), which is stored as a float32 BLOB. At search time the query is embedded and compared with every stored vector by cosine similarity; the top 60 are kept. If Ollama is not running the layer silently returns nothing and lexical search continues.

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|}$$

Web adapter. A standard-library HTTP server bound to 127.0.0.1:8765 calls the C++ binary with argument lists, so the engine remains the single implementation of search. Its endpoints are listed in Table 3.9. A BackgroundIndexer runs the engine in a worker thread, streams output into a lock-protected log, supports cancellation and afterwards labels each document with its extraction method and an estimated page count.

Table 3.9: Endpoints of the local web adapter

Method	Endpoint	Purpose
POST	/api/search	Natural-language or exact-word search with filters
POST	/api/index, /api/index/cancel	Start or cancel background indexing
GET	/api/index/progress	Progress, current step and live log
GET	/api/status, /api/stats, /api/table, /api/document	Library status, statistics, listing, one document with pages
POST	/api/metadata	Edit title, tags and personal notes
POST/GET	/api/semantic/index, /api/semantic/status	Build and monitor the local-AI vector index
POST POST	/api/spotlight/sync, /api/open	macOS Spotlight bridge; open a file
	/api/sql	Developer console; only SELECT, PRAGMA and EXPLAIN accepted

Browser interface. It has three views: Memory Search (search box, interpretation bar, date and format filters, ranked cards with highlighted passage, page number, score explanation and related memories), Library (folder indexing with live progress and a filterable document table) and Developer Console (statistics, local-AI index, Spotlight sync and a read-only SQL workbench).

Security and robustness. Shell injection is avoided by launching tools with argument arrays; hanging tools are stopped by a 120 s timeout, output cap and SIGKILL; memory use is bounded by size limits; symbolic links are never followed; the database is protected by transactions and bound parameters; the server listens on 127.0.0.1 only; and every file is processed inside its own try/catch.

3.6.14Complexity summary

Table 3.10: Time complexity of the main operations

Operation	Time cost	Note
Tokenise a document	O(n)	n = bytes
Add a document to the index	O(n + u log V)	u = unique terms
Exact term lookup	O(log V)	ordered map
Prefix expansion	O(log V + k)	k = matches returned
Infix expansion	O(V × L)	bounded to 8 results
Fuzzy expansion	up to $O(V \times L^2)$, pruned	length filter and early row exit
Phrase check	$O(P_1 \times k \times \log P)$	binary search on positions
BM25 for one candidate	O(m × df)	m matched terms; term frequency needs a posting scan
Graph build	O(N × T)	T = concepts per document (max 120)

CHAPTER 4.
RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of solution

4.1.1 Modern tools used

- **Analysis:** BM25 and edit-distance analysis of retrieval behaviour; per-result score breakdowns printed by the engine; SQL queries in the developer console to inspect stored data.
- **Design drawings and models:** architecture and flow diagrams, Graphviz and Matplotlib for the graph, flowcharts, Gantt chart and result charts.
- **Implementation:** C++20 with CMake, SQLite3, Poppler (pdftotext, pdftoppm, pdfinfo), Tesseract OCR, Python 3, HTML, CSS and JavaScript, Ollama (optional). **Report**
- **preparation and project management:** Markdown and HTML-to-PDF report generation, a weekly Gantt plan and the CMake/CTest build record.
- **Testing, characterization and data validation:** CTest with nine C++ test programs and three command-line workflow tests, a 50-query benchmark script, and manual checks of the web interface.

4.1.2 Automated test results

The project registers 12 CTest groups. The project was rebuilt from the submitted source and all 12 passed, in about 4 seconds in total. The test files contain about 290 assertions. Tests use fixed timestamps for ranking so results are deterministic, and checked-in PDF and PNG fixtures for OCR.

Table 4.1: Automated test groups and what each proves

No.	Test group	What it proves
1	DocumentManagerTests	IDs are unique and stable; metadata refresh clears stale text
2	IngestionTests	Recursive scan, extension filter, hidden files and symlinks skipped, duplicates and changes detected
3	StorageTests	Save, reload, upsert and transaction behaviour in SQLite
4	ExtractionTests	Plain text, PDF text layer, scanned-PDF OCR, image OCR, size limits, missing tools
5	IndexingTests	Tokenisation, positions, prefix and phrase search, removal and re-indexing
6	RetrievalTests	Exact, prefix, infix, fuzzy and phrase routes and their evidence
7	RankingTests	BM25 ordering; title, path, phrase, coverage and recency boosts
8	GraphTests	Concept extraction, common-word filtering, shared-concept ranking, seed exclusion, result limits

9	PipelineIntegrationTests	End to end on the 16-document technical corpus: ingest, extract, save, reload, index, retrieve, rank, graph
1	CommandLineHelp	The binary starts and prints help
0	CommandLineWorkflow	index, then search, then re-index is idempotent
1	CommandLineMediaWorkflow	PDF and image fixtures are extracted and searchable

4.1.3 Sample outputs and a worked example

The following output was produced by the command-line interface on the 25-document sample library. The query “schedulng” contains a one-letter typo; no document contains that word, so the fuzzy route runs and finds “scheduling”.

```
AFTERIMAGE library: 25 documents | 1695 searchable terms
Database: /Users/souryam/AFTERIMAGE/lib.db

Matches for: "schedulng"

1. fragment_retrieval_demo [score 2.76]
     /Users/souryam/AFTERIMAGE/data/sample_documents/fragment_retrieval_demo.txt
     Matched by: fuzzy fragment 'schedulng' matched 'scheduling'
     Graph traversal and scheduling combine breadth first search with a queue. The
scheduling engine helps us find a task scheduler when only part of the word is
remembered. A graph traversal visits nodes and records the path through connected
c...

2. os_notes [score 2.63]
     /Users/souryam/AFTERIMAGE/data/sample_documents/os_notes.txt
     Matched by: fuzzy fragment 'schedulng' matched 'scheduling'
     Process scheduling and memory management.

Related memories (connected by shared concepts):
   - graph_algorithms [graph score 2.60]
       Connected through: graph
       /Users/souryam/AFTERIMAGE/data/sample_documents/graph_algorithms.txt
   - os_process_management_threads_deadlocks [graph score 2.60]
       Connected through: scheduling

/Users/souryam/AFTERIMAGE/data/test_corpus_50/os_process_management_threads_deadlo
cks.md
```

The query “ntersection” (a fragment from the middle of a word) is answered by the infix route:

```
AFTERIMAGE library: 25 documents | 1695 searchable terms
Database: /Users/souryam/AFTERIMAGE/lib.db

Matches for: "ntersection"

1. distributed_transactions [score 2.72]
     /Users/souryam/AFTERIMAGE/data/advanced_test_corpus/distributed_transactions.md
     Matched by: infix fragment 'ntersection' matched 'intersection'
     # Distributed Transactions: 2PC, Quorums, CAP, and PACELC        Two-Phase Commit
(2PC) coordinates an atomic transaction across multiple participants. In the
prepare phase, a coordinator asks each participant to promise that it can commit.
Parti...

Related memories (connected by shared concepts):
   - distributed_consensus_deep_dive [graph score 5.21]
       Connected through: distributed, failure

/Users/souryam/AFTERIMAGE/data/advanced_test_corpus/distributed_consensus_deep_div
e.txt
   - database_indexing_strategies [graph score 4.47]
       Connected through: read, write

/Users/souryam/AFTERIMAGE/data/advanced_test_corpus/database_indexing_strategies.t xt

   - computer_architecture_and_cpu_execution [graph score 4.47]
       Connected through: read, write
/Users/souryam/AFTERIMAGE/data/test_corpus_50/computer_architecture_and_cpu_execut
ion.md

   - garbage_collection [graph score 2.60]
       Connected through: phase
       /Users/souryam/AFTERIMAGE/data/advanced_test_corpus/garbage_collection.md
   - postgresql_mvcc_internals [graph score 2.60]
       Connected through: transaction

/Users/souryam/AFTERIMAGE/data/advanced_test_corpus/postgresql_mvcc_internals.txt
```

For the query “raft leader heartbeat” the ranking engine gave the score breakdown of Table 4.2. The values can be reproduced by hand from the formula in Section 3.6.10: for the top result all three query words are matched, so coverage C = 1 and the coverage boost is 0.5 × 1 = 0.5; the second result matches one of three words, so C = 1/3 and the boost is 0.1667. Both files are new, so the recency term is 0.2.

Table 4.2: Score breakdown for the query “raft leader heartbeat”

Rank	Document	BM25	Title boost	Phrase	Coverage	Recency	Final score
1	raft_consensus	19.8138	1.250	0	0.500	0.2	21.7638
2	distributed_consensus_deep_dive	6.4479		0	0.1667	0.2	6.8145

The memory graph also returned related documents: distributed_transactions (graph score 5.21, connected through “distributed” and “failure”), dbms_normalization_and_relational_schema (4.84, through “candidate” and “must”) and os_process_management_threads_deadlocks (2.23, through “must”). The first is a

meaningful link; the second and third are weak lexical links through generic words, which illustrates the limitation noted in Section 3.6.11.

4.1.4 The 50-query memory benchmark

The script tests/test_50_queries.py runs 50 human-style queries in five groups: vague recollections, different wording, messy conversational queries, PDF and OCR queries, and relationship queries. Its pass condition is that at least one result is returned.

Table 4.3: Benchmark results

Library used	Documents	Searchable terms	Queries with results	Average time per query
Bundled sample and test corpora only	25	1,695	26 of 50	about 4 ms
The author's own library	2,185	54,245	30 of 50	about 60 ms

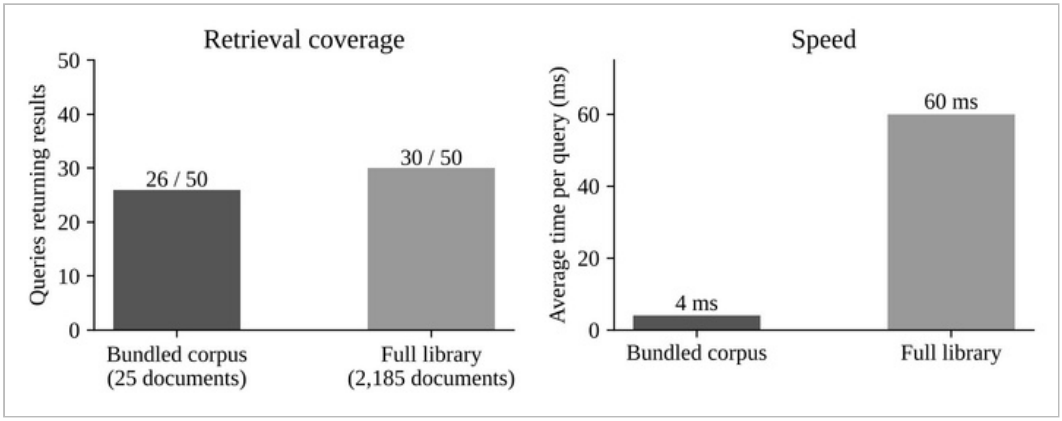


Figure 4.1: Benchmark: queries returning results and average time per query

How these numbers should be read:

- The pass condition only checks that something was returned, not that the correct document is first. A stronger evaluation would label the correct document for every query and report precision at 1 or 3 and mean reciprocal rank (see Section 5.2).
- On the bundled corpus 24 queries return nothing (7, 8, 10, 16, 18, 19, 22 to 26, 29 to 35, 37 to 40, 48 and 49). Most of these ask for scanned PDFs, diagrams, statistics notes or Java stream and RAM/ROM material that exist only in the author's personal library and are not in the submitted bundle.
- Two are genuine misses: query 7 (classes and objects) and query 48 (neural networks) ask about topics that the bundled java_oop_assignment and neural_net_architectures files do contain, so they show where the rule-based query parser needs improvement.
- The author's own library also contains thousands of unrelated files indexed from other folders (for example documentation inside node_modules). This adds noise; for instance "scaled dot product attention" ranks an unrelated documentation page first

and the correct transformer document second. The dependency-noise penalty reduces but does not remove this.

4.1.5 Results in the web interface

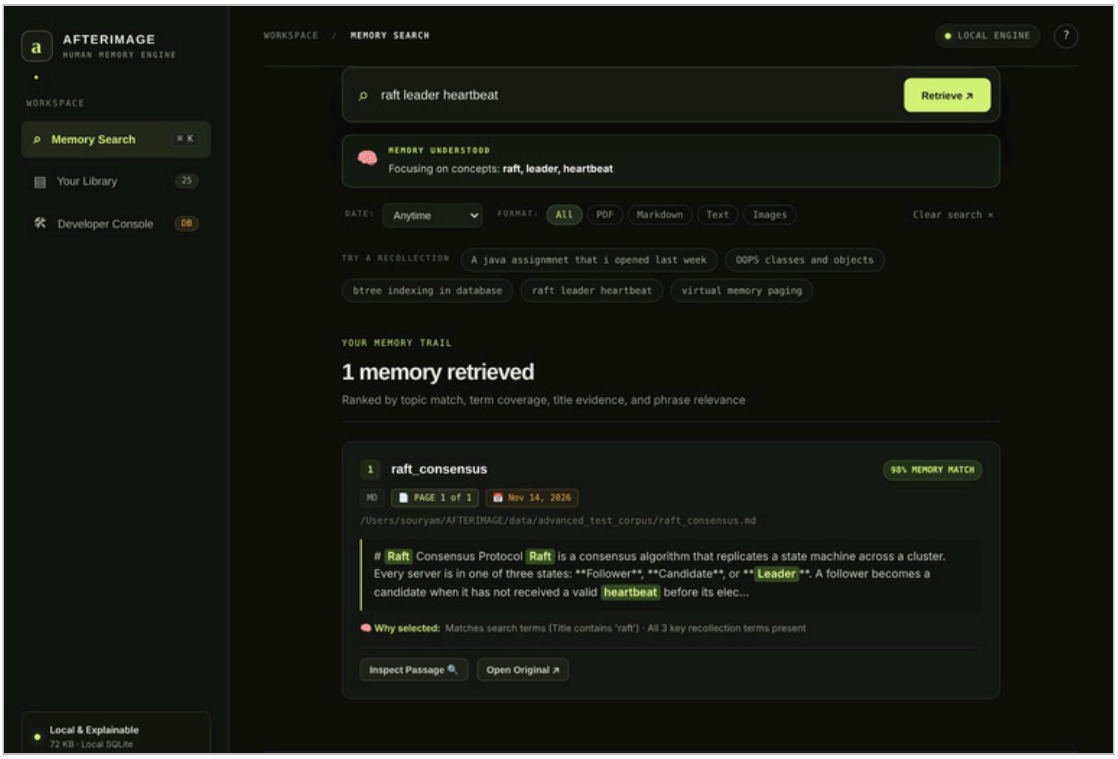


Figure 4.2: Memory Search showing the interpretation bar, 98% match, page, highlighted passage and “Why selected” explanation

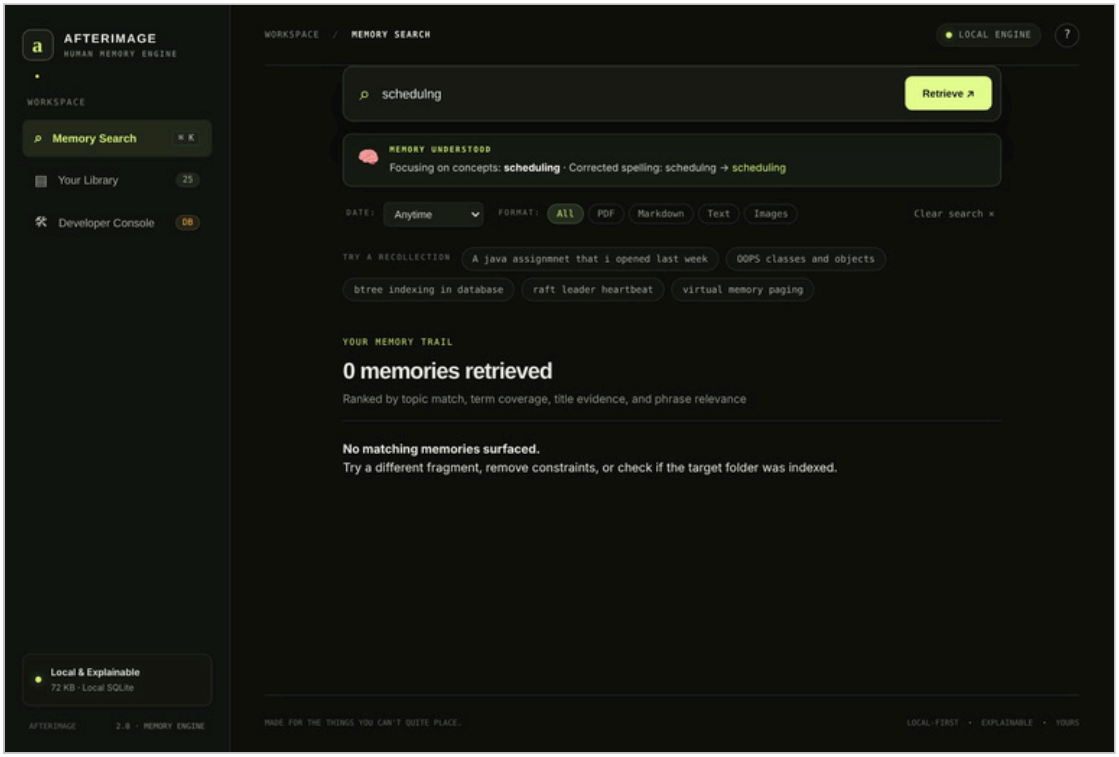


Figure 4.3: Typo-tolerant search: the query “schedulng” is matched to “scheduling”

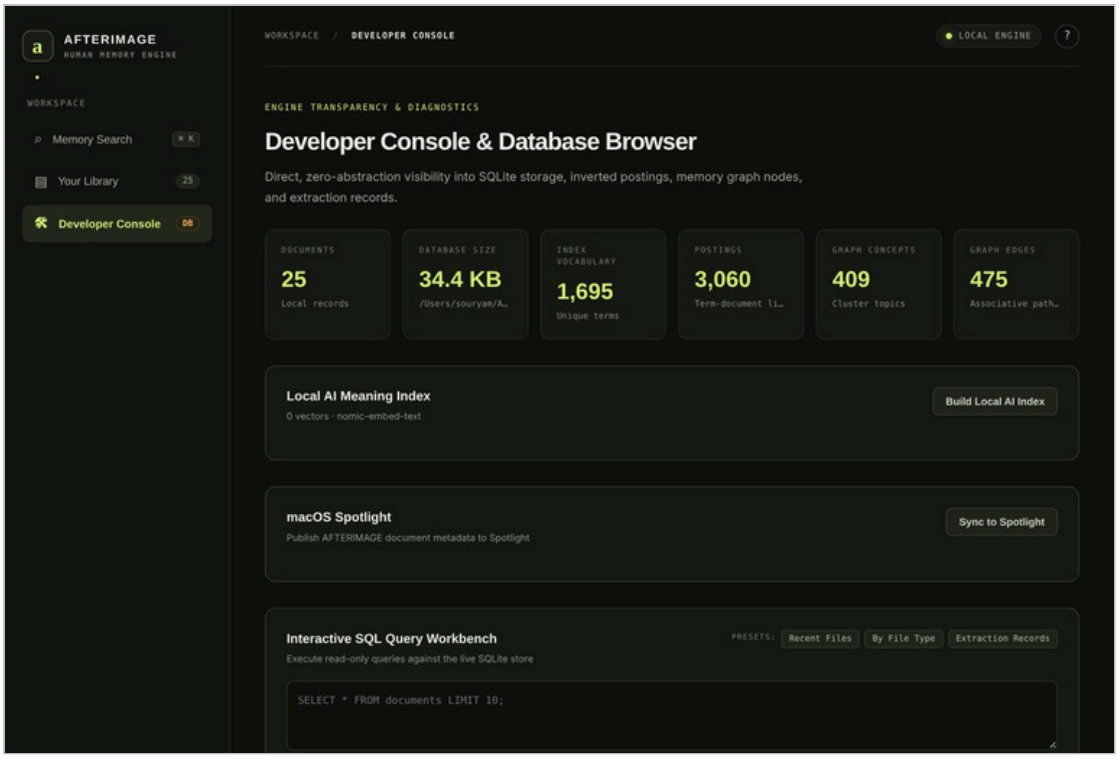


Figure 4.4: Developer Console: library statistics (documents, vocabulary, postings, graph concepts and edges) and tools

4.1.6 Analysis and validation

Table 4.4: Objectives against evidence

Objective	Result	Evidence
O1 stable storage	Met	Storage and document-manager tests pass; IDs survive reload
O2 extraction incl. OCR	Met	Extraction tests and media workflow test pass with PDF and PNG fixtures
O3 positional index with four lookup types	Met	Indexing and retrieval tests pass
O4 typo and fragment queries	Met	Samples “schedulng” and “ntersection” return the right documents
O5 BM25 with breakdown	Met Partly	Table 4.2 reproduced by hand from the formula
O6 related documents	met Partly	Useful link found, but generic words create weak links
O7 natural queries	met	26 of 50 queries answered on the bundled corpus; parser misses on queries 7 and 48
O8 local and fast	Met	No network use; 4 ms and 60 ms average per query

Overall, the retrieval core (objectives O1 to O5 and O8) is fully validated by automated tests and measurements. The natural-language layer (O7) works for the common patterns it was designed for but depends on hand-written rules, so its validation is weaker. Quality was judged against four ISO/IEC 25010 attributes: functional suitability (tests and

benchmark), performance efficiency (milliseconds per query), security (no-shell process launching, localhost-only server, read-only console) and maintainability (eight modules, one test file each).

CHAPTER 5.
CONCLUSION AND FUTURE WORK

5.1. Conclusion

AFTERIMAGE shows how classic data-structure and information-retrieval ideas fit together into a practical tool: a positional inverted index on an ordered map, edit-distance search with pruning, BM25 ranking with transparent boosts, and a concept graph searched with breadth-first search. Around that core, careful engineering (transactions, RAII, no- shell process launching, resource limits, 12 passing test groups) makes it robust, and the browser interface makes it usable. All data stays on the user's machine.

Expected results and outcome. The expected outcome was a system that retrieves a document from typos, stems, middle-of-word fragments and phrases, ranks it with an explainable score, searches inside scans and suggests related documents. These outcomes were achieved and verified (Table 4.4).

Deviation from expected results and reasons.

- Only 26 of 50 benchmark queries return results on the bundled corpus. The main reason is that many queries target personal PDFs and scans that are not in the bundle; two queries are genuine parser misses.
- The index is rebuilt in memory for every search session. This was chosen for simplicity and consistency; it works at the tested scale (60 ms on 2,185 documents) but will not scale to very large libraries.
- The related-document graph is a lexical heuristic and sometimes links documents through generic words.
- The query parser relies on hand-written dictionaries and rules, partly tuned to the benchmark queries, so unseen topics receive less help. The C++ fuzzy route still corrects general one-letter typos.
- OCR quality limits text quality: handwriting, diagrams, low-resolution scans and complex tables are weak, and scanned PDFs stop at 20 pages by default. The tokeniser applies no stemming. The process runner uses POSIX calls, and the launcher and Spotlight bridge are macOS only.

5.2. Future work

1. Persist the inverted index and graph on disk so search starts instantly on large libraries.
2. Add stemming or lemmatisation and BM25F (separate weights for title and body fields).

3. Combine lexical and embedding rankings with reciprocal rank fusion, and build a labelled test set with precision at k and mean reciprocal rank to replace the non-empty- result benchmark.

4. Extract people, places and events to build a real knowledge graph, and read dates from document content.
5. Add handwriting-capable OCR and layout-aware table extraction.
6. Windows support, file-system watchers for automatic re-indexing and optional encryption of the database.

REFERENCES

[1] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333– 389, 2009.

[2] C. D. Manning, P. Raghavan and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.

[3] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[4] K. Spärck Jones, “A statistical interpretation of term specificity and its application in retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[6] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

[7] S. E. Robertson, S. Walker, S. Jones, M. M. Hancock-Beaulieu and M. Gatford, “Okapi at TREC-3,” in *Proc. Third Text REtrieval Conference (TREC-3)*, NIST, 1995.

SQLite

[8] Consortium, “SQLite documentation.” [Online]. Available: https://www.sqlite.org/docs.html

[9] ISO/IEC 14882:2020, *Programming languages — C++*, International Organization for Standardization, 2020.

[10] Poppler developers, “Poppler PDF rendering library and utilities.” [Online]. Available: https://poppler.freedesktop.org

[11] Ollama, “Ollama: run large language and embedding models locally.” [Online]. Available: https://ollama.com

[12]ISO/IEC 25010:2011, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*, 2011.

[13]ISO/IEC 9075, *Information technology — Database languages — SQL*, International Organization for Standardization.

APPENDIX

1. Plagiarism Report

Attach the plagiarism (similarity) report generated for this project report by the university-approved tool on this page. The report cannot be produced from this document; it must be generated and inserted after the final text is approved.

[Plagiarism report to be attached here]

2. Design Checklist

Checklist item	Done	Where addressed
Standards relevant to the design identified	Yes	List of Standards (nine standards)
At least two alternative designs evaluated	Yes	Designs A, B and C compared in Section 3.5
Design constraints identified (regulatory, economic, environmental, health, safety, ethical, social, cost)	Yes	Table 3.2
Features finalised subject to constraints	Yes	Section 3.3
Implementation flowchart / architecture / algorithms provided	Yes	Figures 3.1 to 3.4 and Section 3.6
Modern tools used for analysis, design, testing and report preparation	Yes	Section 4.1.1
Results validated against objectives	Yes	Table 4.4
Limitations, deviations and future work stated	Yes	Chapter 5
Safety of external-tool execution considered	Yes	Section 3.6.4 (no shell, timeout, caps)
Privacy of user data considered	Yes	Local-only processing; localhost server
User manual with step-by-step instructions and pictures	Yes	User Manual

USER MANUAL

(Complete step by step instructions along with pictures necessary to run the project)

Step 1. Requirements

- A macOS or Linux computer (macOS for the Spotlight launcher).
- CMake 3.20 or newer, a C++20 compiler (Clang or GCC) and the SQLite3 development library.
- Poppler (pdftotext, pdftoppm, pdfinfo) and Tesseract OCR for PDF and image text.
- Python 3 for the web interface. Optional: Ollama with the nomic-embed-text model for semantic search.

Step 2. Build and test

```
brew install cmake sqlite poppler tesseract                #macOS; use apt on Linux
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j2
ctest --test-dir build --output-on-failure                 #expect 12 of 12 passed
```

Step 3. Index a folder

Choose a database (optional) and index the sample documents. Re-run the command later: only new or changed files are processed.

```
export AFTERIMAGE_DB=~/AFTERIMAGE/lib.db
./build/afterimage index data                    # 25 documents, 1,695 terms
./build/afterimage index ~/Scans --force --ocr-language hin+eng
```

Step 4. Search from the command line

```
AFTERIMAGE — search your local documents by fragments of memory

USAGE
  afterimage                              Open the interactive search prompt
  afterimage index <folder>               Index a folder (including subfolders)
  afterimage search "some fragment" Run one search and exit

EXAMPLES
  afterimage index ~/Documents
  afterimage
  afterimage search "raft leader heartbeat"
  afterimage search "schedulng" --max-results 5
  afterimage index ~/Scans --force

OPTIONS
  --db <path>              Use a specific SQLite library database
  --force                  Re-extract every known document in the library
  --no-ocr                 Disable OCR for images and scanned PDFs
  --ocr-language <id> Tesseract language, for example eng or hin+eng
  --max-results <N> --     Maximum results to show (1–100)
  phrase --all-terms --    Require an exact consecutive phrase
  exact-only      --no-    Require every query word/fragment to match
  prefix  --no-infix  --   Disable prefix, infix, fuzzy, and phrase expansion
  no-fuzzy        --no-    Disable prefix expansion
  recency                  Disable middle-of-word expansion
  -h, --help               Disable typo-tolerant fallback
                           Rank by content only, ignoring file modification time
                           Show this help

INTERACTIVE COMMANDS
  :help                    Show interactive commands
  :stats                   Show library, index, and graph statistics
  :quit                    Exit AFTERIMAGE

SEARCH RESULTS
  Direct matches are ranked first. Related memories are shown separately
  when the relationship graph finds useful shared concepts.

DEFAULT DATABASE
  ~/.afterimage/library.db (override with --db or AFTERIMAGE_DB)
```

```
./build/afterimage search "schedulng"                    #typo  (fuzzy)  #middle-of-
./build/afterimage search "ntersection"                  word   (infix)  #words  +
./build/afterimage search "raft leaderheartbeat"         phrase + graph
```

Step 5. Start the web interface

```
AFTERIMAGE_DB=~/AFTERIMAGE/lib.db python3 web/server.py
# then open http://127.0.0.1:8765 in a browser
```

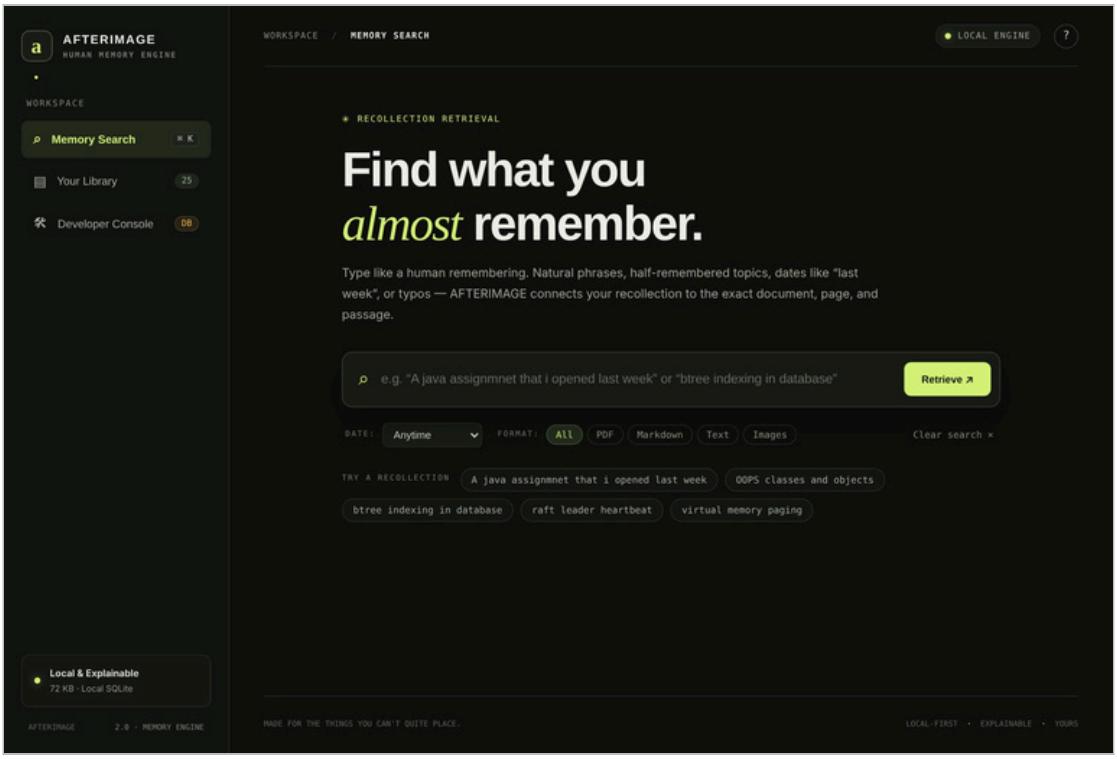


Figure UM.1: Home screen of the web interface with sample recollections to try

Step 6. Search by memory

Type a fragment or a natural sentence (for example “A java assignmnet that i opened last week”) and press Retrieve. The “Memory understood” bar shows how the query was interpreted. Use the Date menu and the format chips (PDF, Markdown, Text, Images) to narrow results. Each card shows the match percentage, page, highlighted passage and why it was selected; use Inspect Passage to read the page text and Open Original to open the file.

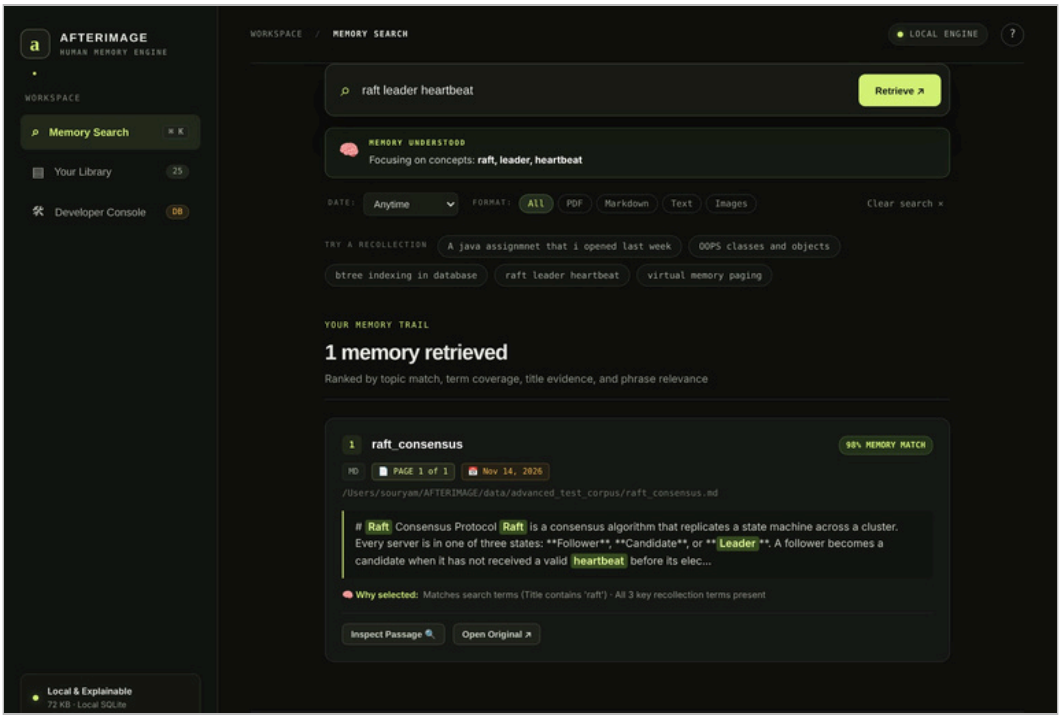


Figure UM.2: Search result card with highlighted passage and explanation

Step 7. Manage the library

Open Your Library to index a folder with live progress, filter the document table and edit titles, tags and notes.

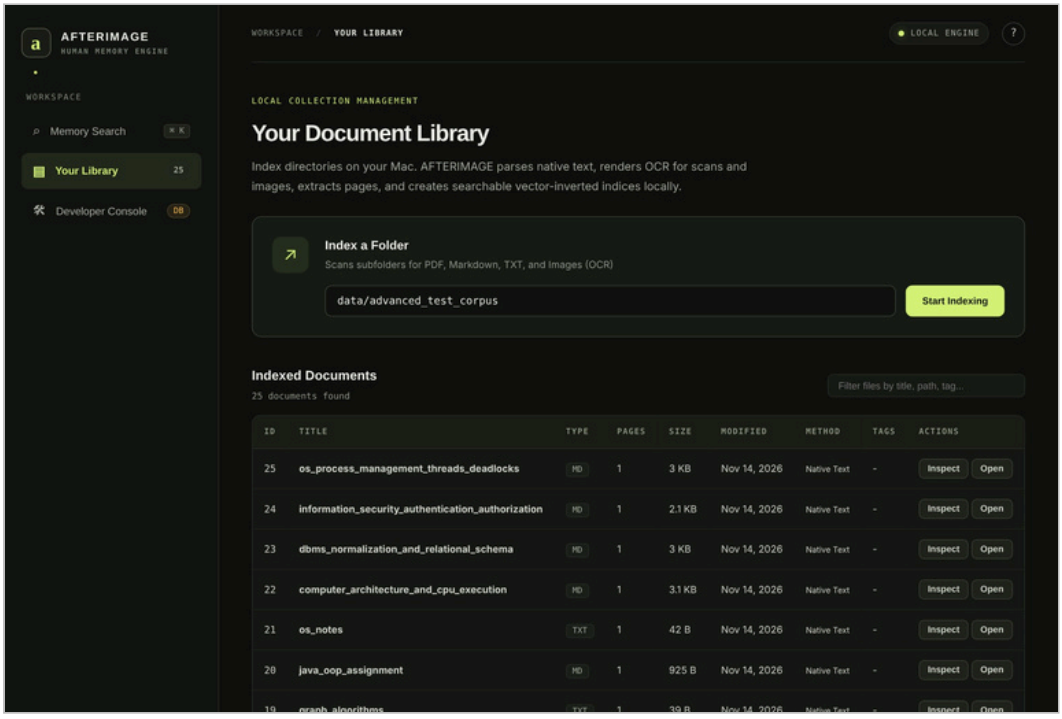


Figure UM.3: Library view listing the indexed documents

Step 8. Developer console (optional)

The Developer Console shows library statistics, builds the optional local-AI index (requires Ollama), syncs metadata to macOS Spotlight and runs read-only SQL (SELECT, PRAGMA, EXPLAIN) against the database.

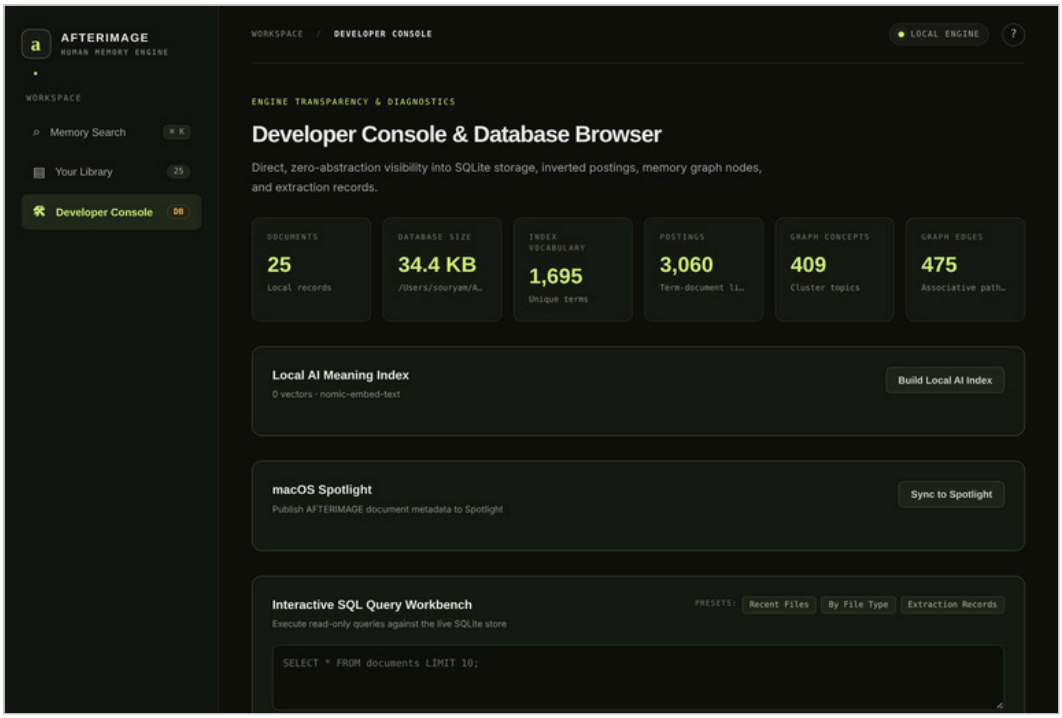


Figure UM.4: Developer console and SQL workbench

Troubleshooting

Table UM.1: Common problems and fixes

Problem	Cause	Fix
Scanned PDF or image returns no text	Tesseract or Poppler not installed	Install them and re-run index with --force
Search finds nothing	Folder not indexed or wrong database	Check AFTERIMAGE_DB and re-run index
No semantic results	Ollama not running or index not built	Start Ollama and use Build Local AI Index
Port 8765 busy	Another server is running	Start with --port 8766
Too many unrelated results	Dependency folders were indexed	Index only your own folders